Subject: ASD&D; Semester: V
Class: TE Batch: T11
Name: Arya Bailur Roll No: 08

EXPERIMENT NO. 07

Aim: To set up and demonstrate infrastructure monitoring using Prometheus / Grafana / Nagios

LO Mapping: LO5

LO5: Implement ML lifecycle management and automated workflows using MLflow, Airflow, and monitoring tools such as Prometheus/Grafana/Nagios.

Theory:

Continuous monitoring refers to an ongoing process of detecting, reporting, and responding to incidents across an organization's infrastructure. After an application has been deployed to production, continuous monitoring ensures that the underlying systems remain healthy and that any anomalies are addressed promptly.

Why Use the Nagios Monitoring Tool?

Below are the key advantages of adopting Nagios for infrastructure monitoring:

- Identifies all categories of network and server problems in real time
- Assists in pinpointing the root cause so that a lasting fix can be applied
- Provides proactive monitoring of the complete infrastructure and business-critical processes
- Facilitates troubleshooting of server performance bottlenecks
- Supports infrastructure capacity planning to prevent failures from aging systems
- Helps maintain both security and high availability of services
- Can automatically remediate certain issues during emergency situations
- Nagios is widely regarded as one of the most popular open-source monitoring platforms. It allows organizations to detect and resolve IT infrastructure issues before they impact essential business operations. Nagios is capable of monitoring applications, services, and the entire IT stack.
- NRPE (Nagios Remote Plugin Executor) is designed to run check plugins on remote machines. On Windows hosts, the equivalent agent is NSClient++, which listens for incoming Nagios queries and performs local checks such as disk usage, CPU load, memory, and running services.
- NSClient++ operates as a Windows service on the monitored host and awaits requests from the Nagios server, typically over the NRPE protocol. When a resource check is requested, NSClient++ executes it locally and sends the result back to Nagios.

Note on Running Nagios on Windows

Nagios Core is developed for Linux and cannot be installed natively on Windows. The recommended approach for Windows users is to run the Nagios Core server inside a Docker container (using Docker Desktop with the WSL2 backend), while monitoring Windows hosts via NSClient++, which serves as the Windows counterpart to NRPE. All commands shown below are

intended for Windows PowerShell / Command Prompt.

1 – Prerequisites: Install Docker Desktop

```
docker --version
```

```
wsl --install
```

```
wsl --set-default-version 2
```

2 – Pull and Launch the Nagios Core Docker Image

Download a pre-built Nagios Core image and start it as a container, mapping the web interface port to the host machine.

```
docker pull jasonrivers/nagios:latest
```

```
docker run --name nagios4 -d -p 8080:80 jasonrivers/nagios:latest
```

```
docker ps
```

3 – Configure the Apache/Nagios Admin Password

Set or reset the password for the nagiosadmin user within the running container:

```
docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
```

4 – Open the Nagios Web Interface

Navigate to the following URL in a browser on the Windows host:

```
http://localhost:8080/nagios/
```

Username: nagiosadmin Password: (the one configured in step 3)

5 – Install NSClient++ on the Windows Remote Host

On the Windows machine to be monitored, download and install NSClient++ (the Windows NRPE equivalent):

```
https://github.com/mickem/nscp/releases/latest/download/NSCP-0.5.2-x64.msi
```

```
Start-Process msiexec.exe -ArgumentList '/i NSCP.msi ADDLOCAL=ALL
MODULES=CheckExternalScripts,NRPEServer /qn' -Wait
```

6 – Configure NSClient++

Edit the NSClient++ configuration file and add the Nagios server's IP to the allowed hosts:

```
notepad nsclient.ini
```

```
allowed hosts=127.0.0.1,192.168.1.100
```

After saving, restart the NSClient++ service:

```
Restart-Service nscp
```

7 – Verify Connectivity from Nagios

From within the Nagios Docker container, run an NRPE check against the Windows remote host to confirm communication (replace 192.168.1.50 with the actual host IP):

```
docker exec -it nagios4 /opt/nagios/libexec/check_nrpe -H 192.168.1.50
```

8 – Register the Windows Host in Nagios

Create a configuration file for the new host inside the Nagios container. A PING service is also defined to continuously verify host reachability:

```
define host {
    use             windows-server
    host_name       Windows_Host_001
    alias           Windows Host 001
    address         192.168.1.50
    register        1
}

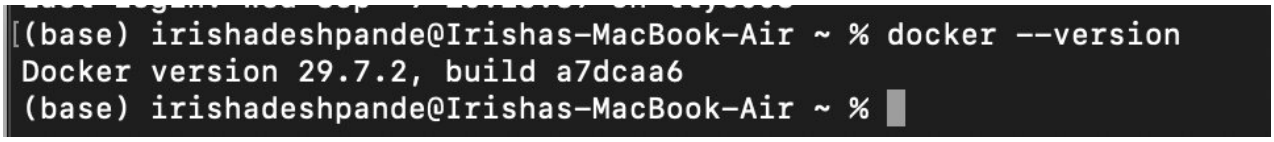
define service {
    host_name             Windows_Host_001
    service_description   PING
    check_command         check_ping!100.0,20%!500.0,60%
    max_check_attempts    2
    check_interval        2
    retry_interval        2
    check_period          24x7
    check_freshness       1
    contact_groups        admins
    notification_interval 2
    notification_period   24x7
    notifications_enabled 1
    register              1
}
```

Validate the configuration for errors, then restart the Nagios container to apply changes.

9 – Verify the Host in the Nagios Web Interface

Open the Nagios web interface (http://localhost:8080/nagios/) and confirm that the new Windows host appears under Current Status → Hosts.

Implementation:



Docker version check

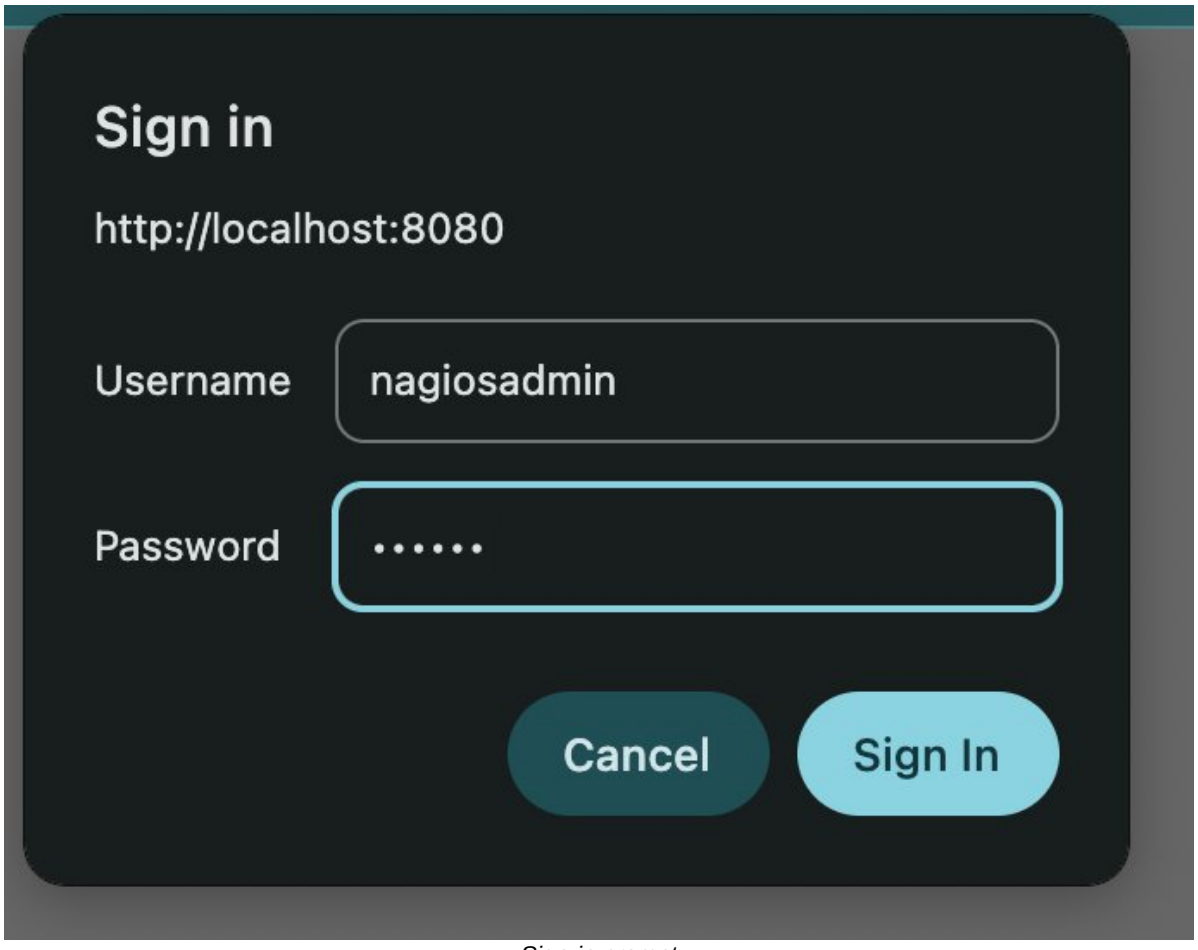
```
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker pull jasonrivers/nagios:latest
latest: Pulling from jasonrivers/nagios
6fe99ef1bed3: Pulling fs layer
c42073aaf047: Pulling fs layer
254354b70ac0: Pulling fs layer
54b9e993e315: Pulling fs layer
80b6ab7aef5d: Pulling fs layer
a80db686d641: Pulling fs layer
6fe99ef1bed3: Pull complete
0d59a81a5608: Pull complete
7a7447982198: Pull complete
c5701b5e1540: Pull complete
1b9771f55a12: Pull complete
a03720c2c16b: Pull complete
e68fcda12c0c: Pull complete
f47c3af6bf44: Pull complete
cb259a83ac3d: Pull complete
fdbaa15b8986: Pull complete
748b632e75e7: Pull complete
9288c4992679: Pull complete
dc3a88012a3e: Pull complete
50ea270d34e9: Pull complete
37f75c8146e2: Pull complete
e4dc1d4e010c: Pull complete
```

Pulling the Nagios Docker image

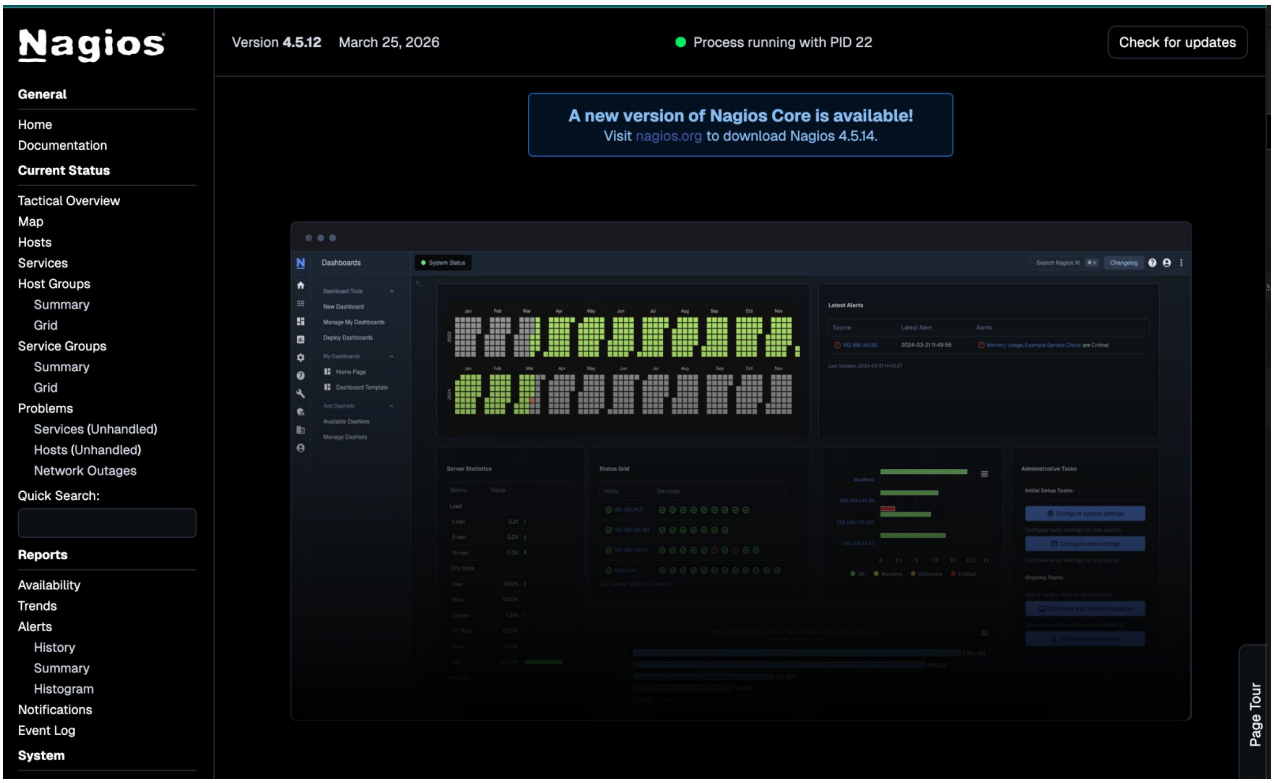
```
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec -it nagios4 htpasswd -c /opt/nagios/etc/htpasswd.users nagiosadmin
New password:
Re-type new password:
Adding password for user nagiosadmin

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
```

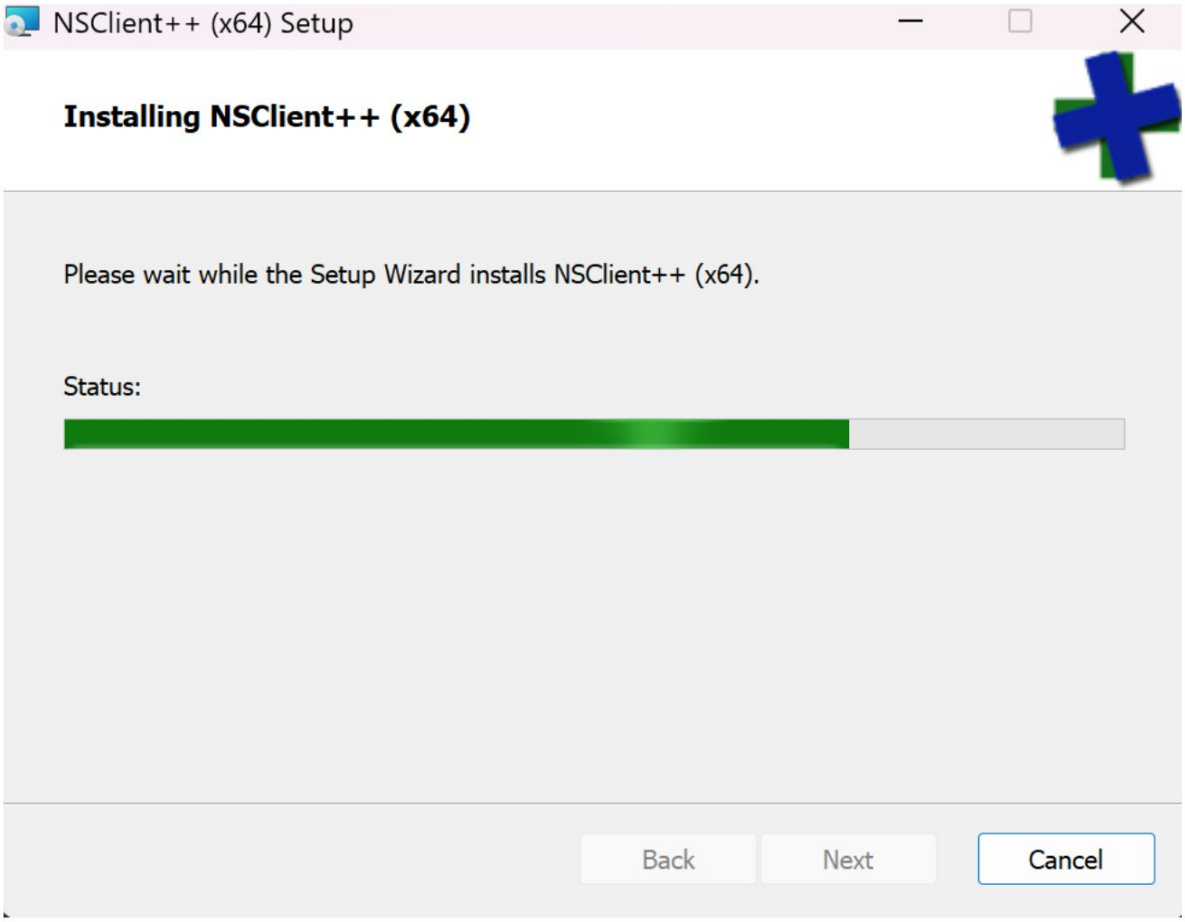
Setting admin password



Sign-in prompt



Nagios web dashboard



NSClient++ installation

```
CheckHelpers = enabled

; Load on startup - If the module should be loaded on startup
CheckExternalScripts = enabled

; Load on startup - If the module should be loaded on startup
CheckNSCP = enabled


; in flight - TODO
[/settings/NRPE/server]

verify mode = peer-cert

tls version = tlsv1.2+

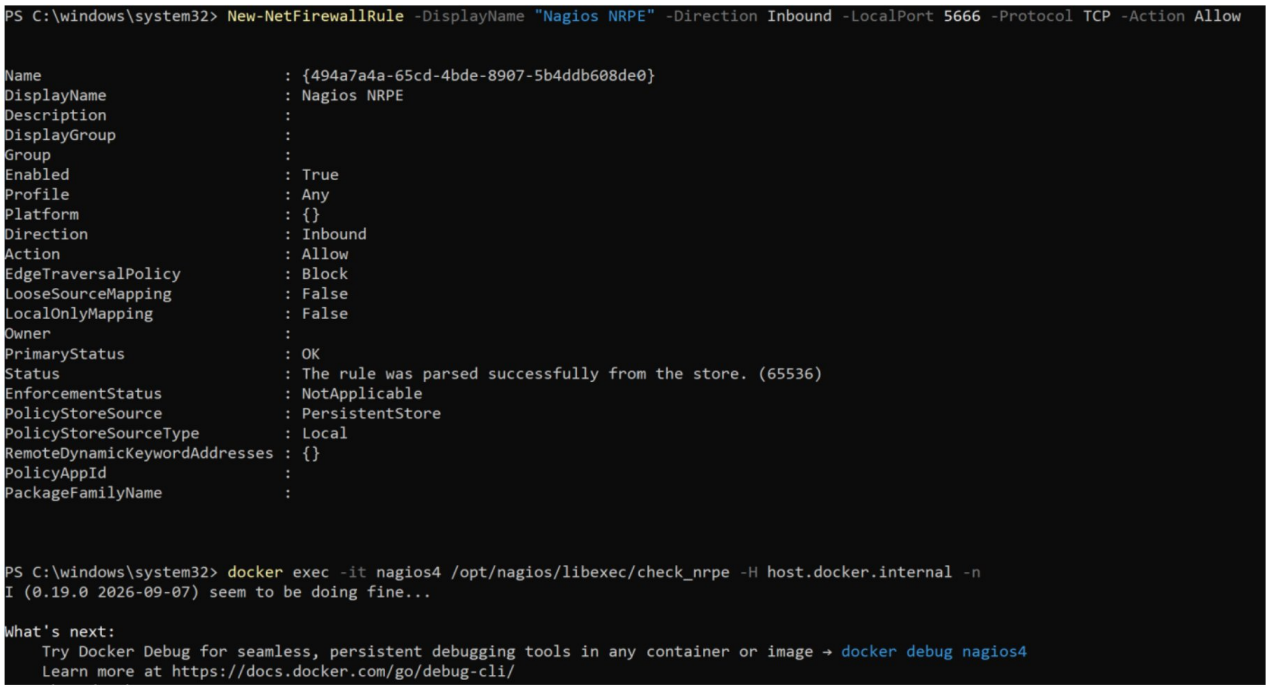
insecure = false


; in flight - TODO
[/settings/default]

allowed hosts = 127.0.0.1,192.168.29.181,172.17.16.1,172.17.0.0/16

password = vOiMR4smBNLkJBxs
```

NSClient++ configuration



Firewall rule and NRPE connectivity check

```
Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
        Checked 7 services.
        Checked 1 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 1 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...
```

Nagios configuration verification (pass 1)

```
Running pre-flight check on configuration data...

Checking objects...
        Checked 7 services.
        Checked 1 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 1 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check

What's next:
    Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug nagios4
    Learn more at https://docs.docker.com/go/debug-cli/
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 getent hosts host.docker.internal
192.168.65.254  host.docker.internal
(base) irishadeshpande@Irishas-MacBook-Air ~ %
```

Nagios configuration verification (pass 2)

```
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 mkdir -p /opt/nagios/etc/servers
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 sh -c 'cat > /opt/nagios/etc/servers/MacBook.cfg
<<EOF
define host {
    use generic-host
    host_name MacBook_Air
    alias MacBook Air
    address 192.168.65.254
    register 1
}

define service {
    host_name MacBook_Air
    service_description PING
    check_command check_ping!100.0,20%!500.0,60%
    max_check_attempts 2
    check_interval 2
    retry_interval 2
    check_period 24x7
    contact_groups admins
    notification_interval 2
    notification_period 24x7
    notifications_enabled 1
    register 1
}
EOF'
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 cat /opt/nagios/etc/servers/MacBook.cfg
define host {
    use generic-host
    host_name MacBook_Air
    alias MacBook Air
    address 192.168.65.254
    register 1
}
```

Host configuration file creation

```
}

define service {
    host_name MacBook_Air
    service_description PING
    check_command check_ping!100.0,20%!500.0,60%
    max_check_attempts 2
    check_interval 2
    retry_interval 2
    check_period 24x7
    contact_groups admins
    notification_interval 2
    notification_period 24x7
    notifications_enabled 1
    register 1
}
```

Host definition displayed

```
Nagios Core 4.5.12
Copyright (c) 2009-present Nagios Core Development Team and Community Contributors
Copyright (c) 1999-2009 Ethan Galstad
Last Modified: 2026-03-25
License: GPL

Website: https://www.nagios.org
Reading configuration data...
   Read main config file okay...
   Read object config files okay...

Running pre-flight check on configuration data...

Checking objects...
        Checked 7 services.
        Checked 1 hosts.
        Checked 1 host groups.
        Checked 0 service groups.
        Checked 1 contacts.
        Checked 1 contact groups.
        Checked 23 commands.
        Checked 5 time periods.
        Checked 0 host escalations.
        Checked 0 service escalations.
Checking for circular paths...
        Checked 1 hosts
        Checked 0 service dependencies
        Checked 0 host dependencies
        Checked 5 timeperiods
Checking global event handlers...
Checking obsessive compulsive processor commands...
Checking misc settings...

Total Warnings: 0
Total Errors:   0

Things look okay - No serious problems were detected during the pre-flight check
```

Configuration validation – no errors

```
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 grep -nE 'cfg_(file|dir)' /opt/nagios/etc/nagios.
cfg
29:cfg_file=/opt/nagios/etc/objects/commands.cfg
30:cfg_file=/opt/nagios/etc/objects/contacts.cfg
31:cfg_file=/opt/nagios/etc/objects/timeperiods.cfg
32:cfg_file=/opt/nagios/etc/objects/templates.cfg
35:cfg_file=/opt/nagios/etc/objects/localhost.cfg
38:#cfg_file=/opt/nagios/etc/objects/windows.cfg
41:#cfg_file=/opt/nagios/etc/objects/switch.cfg
44:#cfg_file=/opt/nagios/etc/objects/printer.cfg
48:# extension) in a particular directory by using the cfg_dir
51:#cfg_dir=/opt/nagios/etc/servers
52:#cfg_dir=/opt/nagios/etc/printers
53:#cfg_dir=/opt/nagios/etc/switches
54:#cfg_dir=/opt/nagios/etc/routers
76:# object configuration files (see the cfg_file and cfg_dir options above).
1347:cfg_dir=/opt/nagios/etc/conf.d
1348:cfg_dir=/opt/nagios/etc/monitor
```

Config file references in nagios.cfg

Current Network Status
Last Updated: Thu Sep 10 08:51:27 UTC 2026
Updated every 90 seconds
Nagios® Core™ 4.5.12 - www.nagios.org
Logged in as *nagiosadmin*

View History For all hosts
View Notifications For All Hosts
View Host Status Detail For All Hosts

Host Status Totals

Up	Down	Unreachable	Pending
1	0	0	1

All Problems	All Types
0	2

Service Status Totals

Ok	Warning	Unknown	Critical	Pending
7	1	0	0	0

All Problems	All Types
1	8

Service Status Details For All Hosts

Limit Results: 100

Host	Service	Status	Last Check	Duration	Attempt	Status Information
MacBook_Air	PING	OK	09-10-2026 08:49:38	0d 0h 2m 58s+	1/2	PING OK - Packet loss = 0%, RTA = 2.17 ms
localhost	Current Load	OK	09-10-2026 08:51:12	0d 0h 20m 15s	1/4	OK - load average: 0.59, 0.31, 0.12
	Current Users	OK	09-10-2026 08:46:55	0d 0h 19m 32s	1/4	USERS OK - 0 users currently logged in
	HTTP	WARNING	09-10-2026 08:50:38	0d 0h 15m 49s	4/4	HTTP WARNING: HTTP/1.1 401 Unauthorized - 695 bytes in 0.020 second response time
	PING	OK	09-10-2026 08:48:21	0d 0h 18m 6s	1/4	PING OK - Packet loss = 0%, RTA = 0.21 ms
	Root Partition	OK	09-10-2026 08:49:04	0d 0h 17m 23s	1/4	DISK OK - free space: / 214959 MiB (98.92% inode=100%):
	Swap Usage	OK	09-10-2026 08:49:47	0d 0h 16m 40s	1/4	SWAP OK - 100% free (1023 MB out of 1023 MB)
	Total Processes	OK	09-10-2026 08:50:30	0d 0h 15m 57s	1/4	PROCS OK: 27 processes with STATE = RSZDT

Results 1 - 8 of 8 Matching Services

Service status details for all hosts

```
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 ping -c 4 192.168.65.254
PING 192.168.65.254 (192.168.65.254) 56(84) bytes of data.
64 bytes from 192.168.65.254: icmp_seq=1 ttl=63 time=0.414 ms
64 bytes from 192.168.65.254: icmp_seq=2 ttl=63 time=0.274 ms
64 bytes from 192.168.65.254: icmp_seq=3 ttl=63 time=0.289 ms
64 bytes from 192.168.65.254: icmp_seq=4 ttl=63 time=0.929 ms

--- 192.168.65.254 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3077ms
rtt min/avg/max/mdev = 0.274/0.476/0.929/0.266 ms
(base) irishadeshpande@Irishas-MacBook-Air ~ % docker exec nagios4 /opt/nagios/libexec/check_ping -H 192.168.65.254
-w 100.0,20% -c 500.0,60%
PING OK - Packet loss = 0%, RTA = 1.09 ms|rta=1.087000ms;100.000000;500.000000;0.000000 pl=0%;20;60;0
```

PING check output from container

Conclusion:

Nagios was successfully deployed as a monitoring solution for tracking hosts and services. The system was configured to execute periodic checks and present the status of all monitored resources through its web interface. This experiment demonstrated that Nagios provides robust and effective monitoring capabilities, enabling early detection of infrastructure and service-level issues.